OS-ORANGE PROBLEM

NAME:DHANUSH K

SRN:PES1UG24CS661

Phase 1: Object Storage Foundation

1A:

```
dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B:

```
dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$
```

Phase 2: Tree Objects

2A:

```
dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$ make test_tree
./test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$
```

2B:

```
dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$ find .pes/objects -type f
.pes/objects/08/2c02496b49f40126d7d6a4912615ad7f0f99a5b770e3814e8c14d2427ea17b
.pes/objects/dd/7493dd11a4e1855394b78721d8c44a16ff6db5efb484e209c638552d65d941
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$ hexdump -C $(find .pes/objects -type f | head -n 1)
00000000  63 6f 6d 6d 69 74 20 31  36 34 00 74 72 65 65 20  |commit 164.tree |
00000010  64 64 37 34 39 33 64 64  31 31 61 34 65 31 38 35  |dd7493dd11a4e185|
00000020  35 33 39 34 62 37 38 37  32 31 64 38 63 34 34 61  |5394b78721d8c44a|
00000030  31 36 66 66 36 64 62 35  65 66 62 34 38 34 65 32  |16ff6db5efb484e2|
00000040  30 39 63 36 33 38 35 35  32 64 36 35 64 39 34 31  |09c638552d65d941|
00000050  0a 61 75 74 68 6f 72 20  50 45 53 20 55 73 65 72  |.author PES User|
00000060  20 3c 70 65 73 40 6c 6f  63 61 6c 68 6f 73 74 3e  | <pes@localhost>|
00000070  20 31 37 37 36 33 32 31  32 33 34 0a 63 6f 6d 6d  | 1776321234.comm|
00000080  69 74 74 65 72 20 50 45  53 20 55 73 65 72 20 3c  |itter PES User <|
00000090  70 65 73 40 6c 6f 63 61  6c 68 6f 73 74 3e 20 31  |pes@localhost> 1|
000000a0  37 37 36 33 32 31 32 33  34 0a 0a 74 65 73 74     |776321234..test|
000000af
dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$
```

Phase 3: The Index (Staging Area)

3A:

```
dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$ ./pes init
echo "hello" > file.txt
./pes add file.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     a.txt
  staged:     file.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  Makefile
  untracked:  index.c
  untracked:  object.c
  untracked:  test.txt
  untracked:  test_objects
  untracked:  README.md
  untracked:  test_tree
  untracked:  test_objects.c
  untracked:  pes.c
  untracked:  commit.c
  untracked:  pes.h
  untracked:  tree.h
  untracked:  tree.c
  untracked:  test_tree.c
  untracked:  commit.h
  untracked:  test_sequence.sh
```

```
Initialized empty PES repository in .pes/
Staged changes:
  staged:     a.txt
  staged:     file.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  Makefile
  untracked:  index.c
  untracked:  object.c
  untracked:  test.txt
  untracked:  test_objects
  untracked:  README.md
  untracked:  test_tree
  untracked:  test_objects.c
  untracked:  pes.c
  untracked:  commit.c
  untracked:  pes.h
  untracked:  tree.h
  untracked:  tree.c
  untracked:  test_tree.c
  untracked:  commit.h
  untracked:  test_sequence.sh
  untracked:  index.h
  untracked:  .gitignore

dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$
```

3B:

```
dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776321234 6 a.txt
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776321471 6 file.txt
dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$
```

Phase 4: Commits and History

4A:

```
dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$ ./pes log
commit 757d68fb55552de229dc14b9f095ab9d07378cd790a55b2cec63296d5b6aa587
Author: PES User <pes@localhost>
Date:   1776331671

    Add farewell

commit 8628cd714dfeee6670885a885f92c6f6ec2dd0bd0782b8f8ea36315e7103beb5
Author: PES User <pes@localhost>
Date:   1776331671

    Add worldell

commit 1e1df1a5283f264e1b87d3d233d7a3068f85aba496b7b0023e6e3a8663dc8a03
Author: PES User <pes@localhost>
Date:   1776331671

    Initial commit

commit 58d67221ff0ec991949ae97737f38e26ab34644a171314bca3e338f79f37b7d2
Author: PES User <pes@localhost>
Date:   1776321859

    two filesommit

commit 50a7629f8f6ac85e4676651dc4020bad690f2a415ebc525c57535bc4bbecc7cb
Author: PES User <pes@localhost>
Date:   1776321649
```

```
    two filesommit

commit 50a7629f8f6ac85e4676651dc4020bad690f2a415ebc525c57535bc4bbecc7cb
Author: PES User <pes@localhost>
Date:   1776321649

    first commitit

commit 082c02496b49f40126d7d6a4912615ad7f0f99a5b770e3814e8c14d2427ea17b
Author: PES User <pes@localhost>
Date:   1776321234

    test2123a
```

4B:

```
dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/08/2c02496b49f40126d7d6a4912615ad7f0f99a5b770e3814e8c14d2427ea17b
.pes/objects/1e/1df1a5283f264e1b87d3d233d7a3068f85aba496b7b0023e6e3a8663dc8a03
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/4f/31c356705a225cfb4416b6b011806a52104e4ff1d0a6029f7d9fab0bceb81a
.pes/objects/50/a7629f8f6ac85e4676651dc4020bad690f2a415ebc525c57535bc4bbecc7cb
.pes/objects/58/d67221ff0ec991949ae97737f38e26ab34644a171314bca3e338f79f37b7d2
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/75/7d68fb55552de229dc14b9f095ab9d07378cd790a55b2cec63296d5b6aa587
.pes/objects/86/28cd714dfeee6670885a885f92c6f6ec2dd0bd0782b8f8ea36315e7103beb5
.pes/objects/dd/7493dd11a4e1855394b78721d8c44a16ff6db5efb484e209c638552d65d941
.pes/objects/e1/292cfb746bbf0d03ee325f81aeec6129b10044052ad86ef03a2ed629b36b24
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

4C:

```
dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$
```

```
ref: refs/heads/main
dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$ cat .pes/refs/heads/main
50a7629f8f6ac85e4676651dc4020bad690f2a415ebc525c57535bc4bbecc7cb
dhanush@dhanush-VirtualBox:~/PES1UG24CS661-pes-vcs$
```

QUESTIONS:

Q5.1: Branching and Checkout

A branch in Git (and PES) is a file in .pes/refs/heads/ that stores a commit hash. To implement pes checkout <branch>, the following steps are required:

1. The .pes/HEAD file must be updated to point to the selected branch, i.e., it should contain ref: refs/heads/<branch>.
2. The commit hash stored in .pes/refs/heads/<branch> must be read.
3. The corresponding commit object should be loaded, and its tree should be retrieved.
4. The working directory must be updated to match this tree. This involves deleting or modifying existing tracked files and recreating files from the tree by reading blob objects.
5. The index must also be updated to match the new tree state.

This operation is complex because it involves safely modifying the working directory, handling nested directory structures, and ensuring that uncommitted changes are not lost. It requires careful state management and file reconstruction.

Q5.2: Detecting Dirty Working Directory Conflicts

To detect conflicts when switching branches, the system must compare three versions of each tracked file: the working directory, the index, and the target branch.

1. First, check if the working directory file differs from the index by comparing metadata such as file size and modification time.
2. If the file is modified, compare the index version with the version in the target branch.
3. If both differ, it indicates a conflict.

A conflict occurs when:

- The working directory file is different from the index (uncommitted changes), and
- The index version is different from the target branch version.

In such cases, the checkout operation must be aborted to prevent data loss.

Q5.3: Detached HEAD

A detached HEAD state occurs when .pes/HEAD contains a commit hash directly instead of a reference to a branch.

In this state, new commits can still be created, but they are not associated with any branch. As a result, these commits become unreachable once the user switches to another branch, and they may eventually be removed during garbage collection.

To recover such commits, the user can create a new branch pointing to the commit using a command like:
git branch <new_branch> <commit_hash>
or
git checkout -b <new_branch>

This ensures that the commits become reachable again.

Q6.1: Garbage Collection and Reachability

Over time, the object store accumulates unreachable objects that are not referenced by any branch.

To remove them, a mark-and-sweep algorithm can be used:

1. Start from all branch heads in .pes/refs/heads/.
2. Traverse each commit recursively, marking all reachable objects (commits, trees, and blobs).
3. Store all reachable object hashes in a hash set for efficient lookup.
4. Scan the entire .pes/objects/ directory.
5. Delete any object whose hash is not present in the reachable set.

A hash set is used because it provides constant-time lookup and is efficient for large repositories.

For a repository with 100,000 commits and 50 branches, approximately 100,000 commit objects, 100,000 tree objects, and several hundred thousand blob objects may need to be visited. This results in around 500,000 to 700,000 objects in total.

Q6.2: Garbage Collection Race Condition

Running garbage collection concurrently with a commit operation is dangerous due to race conditions.

For example:

1. A commit operation creates new objects (blobs, trees, commit) but has not yet updated the branch reference.
2. At the same time, garbage collection scans the object store.
3. Since the new objects are not yet referenced by any branch, GC considers them unreachable and deletes them.
4. The commit operation then tries to update the branch to point to a commit that references deleted objects, leading to repository corruption.

Git avoids this issue using several mechanisms:

- Locking to prevent concurrent modifications during critical operations.
- Atomic updates to ensure references are updated only after all objects are safely written.
- Temporary references to protect newly created objects.

- A grace period before deleting unreachable objects.

These mechanisms ensure that objects are not prematurely deleted and maintain repository consistency.